

Część I

Word2vec

We want vectors for words and documents.

One idea is to use simple encoding schemes like tf (**1-hot encoding**) or tf-idf.

doc1		<i>Ala ma kota.</i>
doc2		<i>Podobno jest kot w butach.</i>
doc3		<i>Ty chyba masz kota!</i>
doc4		<i>But chyba zgubiłem.</i>

	<i>ala</i>	<i>but</i>	<i>chyba</i>	<i>kot</i>	<i>mieć</i>	<i>podobno</i>	<i>ty</i>	<i>zgubić</i>
<i>kot</i>	0	0	0	1	0	0	0	0
<i>ty</i>	0	0	0	0	0	0	1	0
doc1	1	0	0	1	1	0	0	0
doc2	0	1	0	1	0	1	0	0
doc3	0	0	1	1	1	0	1	0

We want vectors for words and documents.

One idea is to use simple encoding schemes like tf (**1-hot encoding**) or tf-idf.

doc1		<i>Ala ma kota.</i>
doc2		<i>Podobno jest kot w butach.</i>
doc3		<i>Ty chyba masz kota!</i>
doc4		<i>But chyba zgubiłem.</i>

	<i>ala</i>	<i>but</i>	<i>chyba</i>	<i>kot</i>	<i>mieć</i>	<i>podobno</i>	<i>ty</i>	<i>zgubić</i>
<i>kot</i>	0	0	0	1	0	0	0	0
<i>ty</i>	0	0	0	0	0	0	1	0
doc1	1	0	0	1	1	0	0	0
doc2	0	1	0	1	0	1	0	0
doc3	0	0	1	1	1	0	1	0

Good point:

- $\vec{d} = \sum \vec{t}_i$

Bad point:

- lots of 0s (sparsity)

One idea would be to represent words by their *contexts*.

doc1 | *Ala ma kota.*
doc2 | *Podobno jest kot w butach.*
doc3 | *Ty chyba masz kota!*
doc4 | *But chyba zgubiłem.*

	<i>ala</i>	<i>but</i>	<i>chyba</i>	<i>kot</i>	<i>mieć</i>	<i>podobno</i>	<i>ty</i>	<i>zgubić</i>
<i>ala</i>	0	0	0	1	1	0	0	0
<i>but</i>	0	1	0	1	0	1	0	0
<i>chyba</i>	0	0	0	1	1	0	1	0
<i>mieć</i>	1	0	1	2	0	0	1	0
<i>podobno</i>	0	1	0	1	0	0	0	0
<i>ty</i>	0	0	1	1	0	0	0	0
<i>zgubić</i>	0	1	1	0	0	0	0	0

tylko jak?

tylko jak?
word2vec

tylko jak? word2vec

- 1 Znajdź duży korpus tekstu
- 2 Tokenizuj go i przerób na małe litery (zazwyczaj niewiele więcej trzeba robić)
- 3 `word2vec -train corpus.txt -output vectors.bin -binary 1 -size 200`
- 4 ...magia ...
- 5 `word-analogy vectors.bin`
 - `berlin : germany warsaw : ?`

- wcześniej znane były inne sposoby reprezentowania słów w sposób ciągły: Latent Semantic Analysis (LSA) i Latent Dirichlet Allocation (LDA)
 - word2vec daje lepsze wyniki (?? — trwają spory)
 - mniej efektywne
- neuronowy model języka to sieć neuronowa, która podaje prawdopodobieństwo następnego słowa
 - np.: 2-warstwowa RNN z gorącą jedynką na wejściu i softmaksem na wyjściu
- ... (Mikolov i in. 2013) przypadkiem odkrywają, że pierwsza ukryta warstwa takiego neuronowego modelu języka ma ciekawe własności
 - (hmmm „z gorącą jedynką na wejściu” – zatoczyliśmy koło do rozwiązania nr 1!)

- (Mikolov i in. 2013) przerabiają neuronowy model języka:
 - w bardzo efektywny sposób (bez GPU!)
 - z dwiema możliwymi architekturami
 - Bag-of-Words odgadujemy słowo na podstawie 4 poprzednich i 4 następnych
 - Skip-Gram odgadujemy kontekst na podstawie słowa (lepsze, ale wolniejsze niż Bag-Of-Words)
 - powstaje otwartoźródłowy word2vec (C), **read the code!**
- powstają klony (i konkurenci) word2vec
 - gensim (Python)
 - także doc2vec, który daje wektory dla zdań/dokumentów
 - DeepLearning4j / Word2Vec (Java/Scala)
 - GloVe (C)
- obecnie powszechne w sieciach neuronowych używanych do tekstu
 - wektory są wstępnie uczone (*pre-trained*), kiedyś popularniejsze
 - ... ale mogą być strojone i poprawiane przez back-propagation!

Część II

Word2vec – an example

Dimension	POLSKA	WARSZAWA	NIEMCY	BERLIN	KASZLE
1	-0.27012292	-0.01835861	-0.19947428	-0.17646527	0.03791444
2	0.04246731	0.14128253	-0.05653929	0.11264855	-0.11071160
3	-0.01523614	0.02119722	-0.00196103	0.06623120	0.05423568
4	0.01894453	0.02140118	-0.11287201	-0.07882947	0.02524782
5	0.02187652	-0.00296781	0.03736672	0.07852688	0.11268105
6	-0.27613056	-0.15175520	-0.11131834	-0.14680400	-0.17330949
7	-0.17428583	-0.03757830	0.11182468	-0.03131958	0.10413620
8	0.00551154	-0.03053217	0.03964536	-0.10319925	-0.08332019
9	-0.14975686	-0.15915465	-0.01598621	-0.15882795	-0.12599191
10	0.07905192	0.20986149	0.10772143	0.24482583	-0.20465963
...	...	...	...	...	...
80	0.04048239	-0.01440109	0.01979086	0.18944547	0.04622392
81	0.09156226	0.06774288	0.16844919	0.15093276	-0.01741069
82	0.11573993	-0.03688228	0.08079311	0.02097924	-0.00673653
83	-0.03794651	0.06576352	-0.02476925	0.00572530	0.01400651
84	-0.01040313	-0.10621540	0.15676951	-0.02861746	-0.00773343
85	0.10139615	0.10926425	0.04289555	0.03876032	-0.03532438
86	0.04548787	0.12962872	-0.11437502	0.16250397	-0.11435584
87	-0.15668288	-0.19117050	0.13806269	-0.02987691	-0.06026419
88	0.04483532	0.04856057	0.01257597	-0.03194113	-0.00808524
89	-0.04717806	0.00098399	0.03608687	-0.00583685	-0.16375898
90	0.05026702	-0.05492868	0.03396593	-0.04093243	0.01010868
91	0.12174397	0.01548969	0.04247248	-0.01612890	0.29853109
92	-0.08365987	0.01967633	-0.07439028	-0.06559996	-0.09436531
93	0.10151470	-0.05795912	0.30262545	0.17032613	-0.18972276
94	0.04033853	0.09991030	0.02675233	-0.04881262	-0.11126099
95	0.01950025	0.01571056	0.03178382	-0.01264122	-0.02187326
96	0.07984348	-0.04931121	-0.07769927	-0.15864071	-0.01326203
97	-0.14140216	-0.01399103	-0.07868610	0.07687283	-0.04551984
98	-0.10021710	-0.17531149	0.07718656	-0.13035789	0.14774007
99	0.06555040	0.07244883	0.12668802	0.07693016	0.03778543
100	-0.07419354	-0.04784723	-0.16127981	-0.08824276	0.04517183

Cosine similarity between word vectors

	POLSKA	WARSZAWA	NIEMCY	BERLIN	KASZLE
POLSKA	1.000	0.405	0.428	0.308	-0.042
WARSZAWA	0.405	1.000	0.010	0.546	-0.178
NIEMCY	0.428	0.010	1.000	0.445	-0.049
BERLIN	0.308	0.546	0.445	1.000	-0.127
KASZLE	-0.042	-0.178	-0.049	-0.127	1.000

Most similar words to the vector obtained by adding the difference
POLSKA-WARSZAWA to a given word

#	NIEMCY		FRANCJA		CZECHOSŁOWACJA		LITWA	
1	BERLIN	0.587	WARSZAWA	0.685	WARSZAWA	0.720	WARSZAWA	0.768
2	NIENCY	0.530	PARYŻ	0.671	WROCŁAW	0.649	WILNO	0.740
3	WIEDEŃ	0.524	BRUKSELA	0.670	WARSZAWA	0.624	LUBLIN	0.703
4	-WARSZAWA	0.522	BERLIN	0.665	PRAGA	0.623	BIAŁYSTOK	0.700
5	WARSZAWA	0.520	LONDYN	0.657	BIAŁYSTOK	0.619	VVARSZAWA	0.694
6	KOWNO	0.519	HAGA	0.651	WAR.SZAWA	0.616	WARSZAWA	0.692
7	PETERSBURG	0.516	GENEWA	0.649	WAIRSZAWA	0.615	WARSZA.WA	0.690
8	KRÓLEWIEC	0.516	WAISZAWA	0.646	WAISZAWA	0.614	GRODNO	0.683
9	CHARKÓW	0.515	PARYŻ	0.644	WARSZAWRA	0.614	-WARSZAWA	0.678
10	RYGA	0.514	WAR.SZAWA	0.641	-WARSZAWA	0.613	WAISZAWA	0.674
11	WIEDELI	0.513	LOZANNA	0.638	LUBLIN	0.613	WASZAWA	0.669
12	PARYŻ	0.512	WNRSZAWA	0.625	WARSZA-WA	0.612	WA.RSZAWA	0.663
13	BERIIN	0.509	PARYŁ	0.625	WARSIZAWA	0.611	WARAZAWA	0.663
14	DREZNO	0.509	WIEDEŃ	0.620	WA.RSZAWA	0.610	WARSZA-WA	0.663
15	HEIDELBERG	0.505	-WARSZAWA	0.619	WARSZWA	0.609	WAREZAWA	0.662
16	WAISZAWA	0.504	WARSZA.WA	0.618	WARSZ.AWA	0.608	ŁOMŻA	0.658
17	CZECHO-SŁOWACY	0.503	WA.RSZAWA	0.617	WARSZAIWA	0.601	WIŁNO	0.656
18	BERUN	0.503	PAIYŻ	0.617	YVARSZAWA	0.593	LWÓW	0.653
19	WIEDEII	0.502	WARSZAWA	0.617	W-WA	0.590	WARSTAWA	0.653
20	BEULIN	0.502	RYGA	0.616	WAORSZAWA	0.590	WAR.SZAWA	0.652
21	MAGDEBURG	0.502	AMSTERDAM	0.611	WNRSZAWA	0.588	WARSZWA	0.652
22	KILONIA	0.501	MOSKWA	0.608	WAR3ZAWA	0.587	ŁUCK	0.649
23	BERLIU	0.501	WASZYNGTON	0.606	KRAKÓW	0.587	WĄRSZAWA	0.649
24	EERLIN	0.501	PRAGA	0.605	26	0.587	WATSZAWA	0.649
25	BUKAREST	0.501	WARSZAWRA	0.605	VLARSZAWA	0.587	ARSZAWA	0.644

see also:

- <https://projector.tensorflow.org/>
- <https://re-research.pl/pl/post/2017-12-09-00101-polskie-lata.html>

Część III

Word2vec – how does magic works?

Word2vec

- ... is like **language model** as we predict a word against the context

Word2vec

- ... is like **language model** as we predict a word against the context
- but we're not interested in the probabilities (!)
- we **will not** use Word2vec as a language model

Word2vec

- ... is like **language model** as we predict a word against the context
- we use $-\log(p)$ as the loss function
- but we're not interested in the probabilities (!)
- we **will not** use Word2vec as a language model

Word2vec

- ... is like **language model** as we predict a word against the context
- we use $-\log(p)$ as the loss function
- but we won't predict the next word $P(w_i | w_1 \dots w_{i-1})$
- but we're not interested in the probabilities (!)
- we **will not** use Word2vec as a language model

Word2vec

- ... is like **language model** as we predict a word against the context
- we use $-\log(p)$ as the loss function
- but we won't predict the next word $P(w_i | w_1 \dots w_{i-1})$
- but we're not interested in the probabilities (!)
- we **will not** use Word2vec as a language model
- but rather predict the word in the "gap": $P(w_i | w_{i-n} \dots w_{i-1} ? w_{i+1} \dots w_{i+n})$

Word2vec

- ... is like **language model** as we predict a word against the context
- we use $-\log(p)$ as the loss function
- but we won't predict the next word $P(w_i | w_1 \dots w_{i-1})$
- but we're not interested in the probabilities (!)
- we **will not** use Word2vec as a language model
- but rather predict the word in the "gap": $P(w_i | w_{i-n} \dots w_{i-1} ? w_{i+1} \dots w_{i+n})$
- or rather $P(w' \text{ is around } w | w)$

Skip-gram and Negative Sampling

pears and two green apples , and onto a
 u_{-2} u_{-1} t u_1 u_2

Skip-gram and Negative Sampling

pears and two green apples , and onto a

u_{-2} u_{-1} t u_1 u_2

positive examples

u^+ t

two apples

green apples

, apples

and apples

negative examples

u^- t

resulting apples

goblin apples

into apples

matter apples

Skip-gram and Negative Sampling

pears and two green apples , and onto a

u_{-2} u_{-1} t u_1 u_2

positive examples

negative examples

u^+ t

u^- t

two apples

resulting apples

green apples

goblin apples

, apples

into apples

and apples

matter apples

Loss function:

$$L(\Theta; U^+, U^-) = -\left[\sum_{u \in U^+} \log P(u|t) + \sum_{u \in U^-} \log(1 - P(u|t)) \right],$$

Skip-gram and Negative Sampling

pears and two green apples , and onto a

u_{-2} u_{-1} t u_1 u_2

positive examples

negative examples

u^+ t

u^- t

two apples

resulting apples

green apples

goblin apples

, apples

into apples

and apples

matter apples

Loss function:

$$L(\Theta; U^+, U^-) = -\left[\sum_{u \in U^+} \log P(u|t) + \sum_{u \in U^-} \log(1 - P(u|t)) \right], \quad P(u|t) = \frac{1}{1 + e^{-\left(\vec{w}_t \vec{c}_u\right)}}$$

$\Theta = (\vec{w}_1, \dots, \vec{w}_{|V|}, \vec{c}_1, \dots, \vec{c}_{|V|}), \dots$ but $\vec{c}_i$ are discarded after the model is trained

read the source code:

<https://github.com/dav/word2vec/blob/master/src/word2vec.c!>